

CLOCK GENERATOR Using Verilog

ARNAV BANSAL

Presented here is a clock generator design using Verilog that is simulated using ModelSim software. A clock generator is a circuit that produces a timing signal (known as clock signal and behaves as such) for use in synchronising a circuit's operation. The signal can range from a simple symmetrical square wave to more complex arrangements. Basic parts that all clock generators share are a resonant circuit and an amplifier.

In VLSI domain, while designing Verilog code we also need to design test benches to automate the

process of testing. Clock generators are used in test benches to provide a clock signal for testing the model of a synchronous circuit. The clock generator can be used for a number of applications and also for producing symmetric waveforms.

To design the clock generator for a specific application we can design the clock generators using different looping mechanisms available in Verilog language. In Verilog hardware description language, we have different looping statements available. For example, forever loop causes unconditional repetitive execution of statements and simultaneously we need to make use of disable keyword, as it is the method for disabling the clock signal after some time. You need to provide the delay for this.

Software

The following sample code, written in Verilog, produces symmetric waveforms using simulation. There are a number of simulation tools

available; we have used ModelSim SE edition by Mentor Graphics. In the following code you would see that forever loop executes until simulation terminates, and it also tells you how the initial behavior may continue for the duration of a simulation without expiring. Disable statement terminates execution after 350 time steps by disabling clk_loop block.

The sample code given below is for understanding the concept of the clock generator.

```
//parameter halfcycle=50;
parameter stoptime=350;
initial
begin: clk_loop // clk_loop basically
contains the block of statements
    clk=0;
    forever
        begin
            #halfcycle clk =1;
            #halfcycle clk =0;
        end
    end
initial
    #350 disable clk_loop; //
```

In many situations, loops can be constructed using four basic looping mechanisms, but some specific EDA synthesis tools will synthesise only the for loop.

One important note while designing Verilog code is that, always and forever keywords are not interchangeable. But both are cyclic in execution.

Forever loop executes only when it is inside the sequential activity flow, whereas always behaviour becomes active and executes only at the very beginning of the simulation. At the end, we need to disable or terminate the activity flow to the statement as soon as the task is completed.

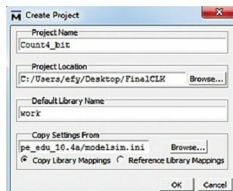


Fig. 1: Create Project window

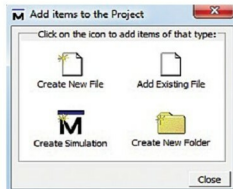


Fig. 2: Add items to the Project window

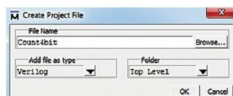


Fig. 3: Create Project File

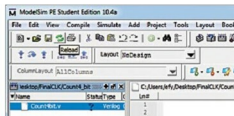


Fig. 4: Workspace window

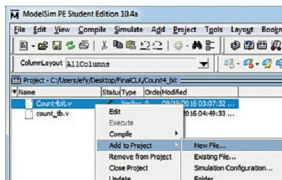


Fig. 5: Adding new file